

Sistemas Operativos

Tarea 2

Torneo de Komikan Concurrente

Ailyn Machuca

Benjamin Cerda

3 de noviembre de 2026

Índice

1. Diseño de la sincronización **1**

1.1. Servidor y emparejamiento 1

1.2. Tablero y movimientos 1

1.3. Recuperación ante fallos 1

1.4. Señal de interrupción y cierre 1

2. Riesgos de concurrencia **2**

2.1. Deadlock 2

2.2. Livelock 2

2.3. Starvation 2

3. Evaluación **2**

1. Diseño de la sincronización

Hicimos todo en un proceso con hilos (`pthreads`), así que no necesitamos memoria compartida de System V. Hay un `main` (servidor), `N` jugadores, un juez por mesa activa y 13 fichas por partida (1 puma y 12 perros).

1.1. Servidor y emparejamiento

Cada jugador que llega se anota en una cola protegida por `mcola` y hace `sem_post(&avisos)`. El servidor espera en `sem_wait(&avisos)` y despierta cuando llega alguien. Con dos jugadores, el primero juega con el puma y el segundo con los perros. El semáforo `mesas_libres` parte en `M` y limita las mesas activas; una pareja espera ahí hasta que un juez libera una. Cada jugador espera el resultado en su semáforo `listo`.

1.2. Tablero y movimientos

Cada partida tiene un mutex para el tablero, las capturas, los movimientos y el estado final. Cada ficha toma el mutex, anota su posición, elige un movimiento y lo suelta; duerme entre 50 y 300 ms, y vuelve a tomarlo para llamar a `mover()`, que valida la jugada. Se comprueba que siga en su casilla, que el destino esté vacío y que el movimiento o salto respete las líneas. Si otra ficha ocupó el destino, se rechaza y queda en el log. Validación y escritura ocurren con el mutex tomado, así que solo una ficha gana una casilla; en `mover()` también se revisa si alguien ganó.

Al terminar se hace `sem_post(&p->fin)`. El juez duerme ahí con `sem_timedwait`: despierta al terminar la partida, si le avisan de una caída o cada 250 ms para revisar latidos (ver 1.3). Al terminar, hace `join` de las fichas, destruye el mutex y el semáforo, avisa a los jugadores y libera la mesa. En `torneo.log`, cada movimiento y su tablero se escriben en una sola toma de `mlog`, sin intercalar líneas de otras mesas. En consola, el tablero se imprime entre `flockfile(stdout)` y `funlockfile(stdout)`; además, `stdout` usa buffer por línea (`setvbuf` con `_IOLBF`) para no perder salida si el programa muere de golpe.

1.3. Recuperación ante fallos

Con `prob_falla`, una ficha se muere de golpe (`pthread_exit`) sin avisar. Según `modo_falla`, cae con el mutex tomado (0), después de soltarlo (1) o al azar (2). Se simula a lo más una caída por partida para comparar las simuladas con las detectadas. En ambos casos el juez declara el abandono.

Si cae con el mutex tomado, este es robusto (`PTHREAD_MUTEX_ROBUST`): el siguiente hilo recibe `EOWNERDEAD`, lo repara con `pthread_mutex_consistent`, anota en `p->caida` la ficha que estaba en `p->dueno` y despierta al juez con `sem_post(&p->fin)`. Sin esto, las demás fichas quedarían bloqueadas.

Si cae sin el mutex, se detecta por latido. Cada ficha anota la hora con reloj monotónico al tomarlo; el juez despierta cada 250 ms, toma el mutex y revisa que ninguna ficha viva lleve más de `rmax + 1 s` sin latido. Al cerrar la partida también revisa que cada ficha haya marcado su salida normal (campo `salio`); así queda registrada una caída ocurrida justo al terminar, aunque no cambie el resultado.

1.4. Señal de interrupción y cierre

El handler se instala con `sigaction`. `SIGINT` queda bloqueada en todos menos `main`; los hilos heredan la máscara con `pthread_sigmask`, así que el handler siempre corre en `main`. Solo pone `parar = 1` y hace `sem_post` (async-signal-safe) en `avisos`, `mesas_libres` y `cierre`, guardando y restaurando `errno`.

Los jugadores que aún no llegan no están durmiendo: esperan en `cierre` con `sem_timedwait`. El handler los despierta; llegan al tiro, el servidor los rechaza y el cierre no tiene que esperarlos. Los que ya estaban en la cola también se rechazan y las partidas activas se cancelan. Al final se hace `join` de todos los hilos, se ignora `SIGINT` antes de destruir los semáforos que usa el handler, se libera la memoria y se cierra el log.

Se revisa que funcionen los `malloc`, los `pthread_create`, los `sem_init` y la creación del mutex de cada partida; si esta falla, se cancela esa partida y se libera la mesa. Si `sem_wait` se interrumpe con `EINTR`, se reintenta; otro error termina el programa.

2. Riesgos de concurrencia

2.1. Deadlock

Con un mutex por casilla, el salto del puma necesitaría tres casillas y dos fichas podrían esperarse entre ellas; por eso usamos uno por tablero. Los mutex se toman en orden `mmesas` → partida → `mlog`, y ninguna ficha duerme con uno tomado. El lock de stdout siempre es el último que se toma: hay printf con el mutex de la partida o con mcola tomado, pero con stdout tomado nunca se pide otro candado, así que no puede formar un ciclo. El mutex robusto resuelve la muerte de una ficha mientras lo tiene tomado.

2.2. Livelock

Las fichas podrían moverse para siempre sin que nadie gane, o pelear siempre por la misma casilla. Lo segundo es muy poco probable porque los retardos son aleatorios. Para lo primero hay un límite de movimientos (`max_movimientos`): al llegar al límite la partida es empate y la mesa se libera.

2.3. Starvation

Cada ficha suelta el mutex al tiro y luego duerme, así que las demás tienen su oportunidad. El juez también lo toma cada 250 ms, pero solo por un instante, sin quitarles turnos a las fichas. Las parejas salen de la cola en orden de llegada y el servidor espera mesas, así que ninguna pareja se queda esperando para siempre.

3. Evaluación

Elegimos lo más simple que funcionara bien: un mutex por tablero evita ordenar candados por casilla. En cada mesa se mueve una ficha a la vez, pero la sección crítica (validar, mover y escribir) es corta frente a los 50–300 ms de espera; las mesas sí juegan en paralelo. Todas las esperas son con semáforos. La única espera periódica es la del juez cada 250 ms para revisar latidos, que cuesta muy poca CPU.

Limitaciones:

1. **Latido:** el margen de `rmax + 1 s` es un compromiso. Más chico detecta antes, pero podría confundir una ficha lenta (por ejemplo, bajo valgrind) con una caída; más grande tarda más en declarar el abandono.
2. **Errores graves:** al ser hilos de un proceso, un segmentation fault en una ficha bota todo el programa. Para aguantarlo habría que usar procesos con `fork`.
3. **Semilla:** fija la llegada de jugadores y las decisiones de cada ficha (cada hilo tiene su semilla para `rand_r`), pero no da reproducibilidad total: el orden en que las fichas consiguen el mutex lo decide el scheduler.